👩‍💻 Guía del estudiante — Hackea y protege CampusShop

Modalidad: en parejas · **Duración:** 2 sesiones de 2 horas **Regla de oro de la unidad:** *toda entrada es hostil hasta que se demuestre lo contrario.*

Cómo se trabaja

El taller tiene tres fases:

1. 🔴 **Atacar** — explotas cada vulnerabilidad y capturas su *bandera* (`FLAG{...}`).
2. 🟢 **Defender** — corriges el código con la defensa vista en clase.
3. ✔️ **Verificar** — `pytest -v` debe pasar de 13 pruebas en rojo a **las 16 en verde**.

Para cada reto documenta en tu informe: **payload usado**, **bandera**, **causa raíz** y **cómo lo corregiste**.

> Las pistas están graduadas. Intenta primero sin mirar; abre la pista solo si te atascas.

🔴 Fase 1 — Ataque

Reto 01 · Inyección SQL — *entra como `admin`*

El login arma la consulta concatenando tu texto. Entra como administrador **sin saber la contraseña**. Captura también los datos de otras tablas desde el **buscador**.

<details>
<summary>💡 Pista 1</summary>
</details>
<details>
<summary>💡 Pista 2</summary>
</details>

Bandera: visible en el panel `/admin` una vez dentro.

Reto 02 · XSS — *roba la "cookie" en una reseña*

El buscador refleja tu texto sin escapar y las reseñas se guardan con `|safe`. El mensaje de la URL (`?msg=`) se pinta con `innerHTML`.

<details>
<summary>💡 Pista</summary>
</details>

Bandera conceptual: `FLAG{xss_ejecuta_js_de_otros}` — demuéstralo con una captura del `alert`.

Reto 03 · Inyección de comandos — *lee `flag.txt`*

En `/admin` → "Diagnóstico de red", el servidor ejecuta `ping` con tu texto y `shell=True`.

<details>
<summary>💡 Pista</summary>
</details>

Bandera: el contenido de `flag.txt`.

Reto 04 · IDOR — *mira la factura de otro*

`GET /api/facturas/{id}` entrega la factura por su ID sin comprobar el dueño. Entra como `ana` y busca la factura de `luis`.

▶ 💡 Pista

Bandera: campo `nota` de la factura 1003.

Reto 05 · Falta de idempotencia — *conviértete en VIP*

El cupón `BIENVENIDA` da +100 de saldo **una sola vez**… pero la comprobación y el registro no son atómicos. Con peticiones simultáneas lo canjeas varias veces. Llega a 250 de saldo para ser VIP.

▶ 💡 Pista

Bandera: campo `premio` de `/api/saldo`.

Reto 06 · CSRF — *cambia el correo de la víctima*

`POST /perfil/email` no exige token anti-CSRF y la cookie no tiene `SameSite`. Abre la página del atacante mientras tienes sesión iniciada.

▶ 💡 Pista

Bandera conceptual: `FLAG{csrf_sin_token}` — adjunta el antes/después del correo.

Reto 07 · Autenticación débil — *rompe los hashes*

Las contraseñas se guardan en **MD5 sin sal**. Con los hashes que sacaste en el reto 01, recupéralos con un diccionario.

▶ 💡 Pista

Bandera: la nota privada de `maria` (visible en su perfil al entrar con la contraseña recuperada).

Reto 08 · SSRF — *alcanza el servicio interno*

"Vista previa de enlace" descarga cualquier URL que le des, incluido el servicio de metadatos interno que solo el servidor puede ver (`http://127.0.0.1:9001/`).

▶ 💡 Pista

Bandera: campo `Nota` de las credenciales robadas.

Reto 09 · Path traversal — *lee el `.env`*

`GET /descargas?f=` concatena tu nombre de archivo sin validar.

▶ 💡 Pista

Bandera: valor de `FLAG` dentro del `.env`.

Reto 10 · Configuración y datos expuestos — *haz hablar a los errores*

El modo debug muestra trazas, el CORS es `*` y hay un secreto escrito en el código.

▶ 💡 Pista

Bandera conceptual: `FLAG{config_insegura}` — señala la traza y el secreto en tu informe.

🟢 Fase 2 — Defensa

Corrige **cada** vulnerabilidad en el código de `campusshop/`. Guía rápida de defensas (todas salen de la presentación de clase):

#	Defensa principal
01	Consultas parametrizadas (`?`), nunca concatenar texto en SQL
02	Escapar la salida (quitar `
03	Sin shell: argumentos como lista (`["ping","-c","1",host]`) + validar que `host` es una IP
04	Autorizar cada objeto: la factura debe ser del usuario autenticado
05	Restricción `UNIQUE` + manejar el error de inserción (sin "comprobar y luego insertar")
06	Token anti-CSRF en el formulario + cookie `SameSite=Lax`
07	Hash lento con sal (bcrypt/Argon2/scrypt), regenerar la sesión al entrar
08	Lista blanca de dominios + bloquear IP privadas y de metadatos
09	Resolver la ruta y comprobar que empieza por la carpeta base
10	`debug=False`, CORS a orígenes concretos, secreto fuera del código, cabeceras de seguridad

✔️ Fase 3 — Verificación

```
pytest -v
```

No has terminado hasta que **las 16 pruebas estén en verde** y la tienda siga funcionando (puedes navegar, iniciar sesión, buscar y canjear el cupón *una* vez).

Ronda extra (opcional): intercambio entre parejas

Intercambia tu carpeta `campusshop/` ya corregida con otra pareja e intenta romper sus parches. Si encuentras un bypass, documéntalo: vale puntos.

📦 Entregables

1. **Informe PDF** con una tabla por vulnerabilidad: payload, bandera, causa raíz, fix y la diapositiva de clase relacionada.
2. **Repositorio Git** con **un commit por vulnerabilidad corregida**, mensajes descriptivos (no `fix`, `update`, `asdf`).
3. Salida de `pytest -v` con las 16 pruebas en verde (captura o texto).

4. Capturas de las banderas capturadas en la Fase 1.

🏆 Evaluación

Criterio	Peso
Explotación de los 10 retos (banderas)	30 %
Mitigación correcta (16 pruebas en verde + revisión de código)	40 %
Informe (claridad, causa raíz, relación con las diapositivas)	20 %
Ronda de intercambio entre parejas	10 %